

DynBench 一周协调执行方案

目标

在一周内完成约 100 个 DynBench 数据集上的 agent benchmark，比较不同模型和 harness 的表现，并保留完整日志以支持复现、失败追踪和后续报告。

本方案强调三个原则：

- 固定数据，测 agent 行为。数据集生成后不再改 seed；多次运行只用于测 agent 随机性和 harness 稳定性。
- 主实验覆盖面优先，敏感性分析只在子集上做。
- 每次运行都必须保留机器可读日志、命令、commit、环境和输出目录，不能只保留最终分数。

当前运行入口

当前 DynBench 正式批量入口是：

```
benchmark/dynbench/run_batch_agents.py
```

单场景 debug 入口是：

```
benchmark/dynbench/run_dynbench.py
```

不要直接用 `cytobridge_agent.cli run` 跑 DynBench。DynBench runner 会额外完成干净 workspace、TASK.md 注入、隐藏 ground truth、public verifier、官方评分和日志保存。

实验分层

主实验

主实验只做单次 repeat，目标是覆盖 100 个数据集和主要 agent 配置。

建议 7 个 arm：

arm_id	模型	harness	目的
gpt_codex	GPT-5.5	Codex harness	GPT 基线

arm_id	模型	harness	目的
deepseek_cc	DeepSeek	CC harness	非 OpenAI 模型基线
mimo_cc	MiMo	CC harness	小米模型基线
gpt_cellcompass	GPT-5.5	CellCompass harness	主力方法
deepseek_cellcompass	DeepSeek	CellCompass harness	harness 泛化
mimo_cellcompass	MiMo	CellCompass harness	harness 泛化
biomni	BioMNI	BioMNI/native	外部 agent 基线

主实验规模：

$100 \text{ datasets} \times 7 \text{ arms} \times 1 \text{ repeat} = 700 \text{ runs}$

按每 run 约 15 分钟估计：

$700 \times 15 \text{ min} = 10500 \text{ min} = 175 \text{ 小时串行}$
1 张 GPU 卡可并行 2 到 4 个 run
5 条并行 lane 时约 35 小时
考虑失败重跑、API 排队和人工排查，预计 2.5 到 4 天

稳定性分析

稳定性分析只在子集上做，不要全量 repeat。

建议：

$20 \text{ datasets} \times \text{GPT-5.5 CellCompass harness} \times 5 \text{ repeats}$

主实验已经包含 1 次，因此额外运行：

$20 \times 4 = 80 \text{ runs}$

如果时间允许，再加一个小规模 provider 对比：

$10 \text{ datasets} \times 3 \text{ arms} \times 2 \text{ extra repeats} = 60 \text{ runs}$

总规模建议控制在：

700 主实验 + 80 稳定性 + 60 小规模复核 + 10% 重跑 buffer $\approx$ 920 runs

5 条并行 lane 时理论运行时间：

$$920 \times 15 / 5 = 46 \text{ 小时}$$

一周内可完成，但需要严格日志管理和每日监控。

并行资源假设

单个 DynBench run 约 15 分钟。根据当前任务规模，一张 GPU 卡预计可以同时跑 2 到 4 个 run：

保守设置：parallel=2

常规设置：parallel=3

激进设置：parallel=4

推荐先用 pilot 测每张卡的稳定并发：

- 如果显存、CPU、IO、API 都稳定，使用 parallel=3 。
- 如果 GPU 利用率低且没有 timeout 增加，可提升到 parallel=4 。
- 如果出现训练变慢、CUDA OOM、日志卡住或 provider rate limit，降到 parallel=2 。

多操作员并行时不要只按人头估算吞吐，而要按 GPU 卡数估算：

$$\text{总并发 lane} \approx \text{GPU 卡数} \times \text{每卡并发数}$$

示例：

GPU 卡数	每卡并发	总并发 lane	920 runs 预计纯运行时间
1	2	2	约 115 小时
1	4	4	约 58 小时
2	3	6	约 38 小时
2	4	8	约 29 小时
3	3	9	约 26 小时

实际完成时间还要加上 API 排队、失败重跑、人工排查和日报整理。计划中按 2 到 4 天完成主运行是合理的。

数据集设计

目标是约 100 个固定数据集。建议按难度和陷阱分层：

类型	数量	目的
easy/sanity	20	检查 pipeline、schema、明显动力学
medium/core	40	主排行榜核心
hard	25	多 fate、多 gene、holdout 更难
adversarial	15	growth confounder、GRN confounder、time-axis、weight/mass trap

每个数据集必须有：

- TASK.md
- train.h5ad
- prediction_targets.json
- fate_classifier.pkl
- hidden ground truth 目录
- 数据生成 config
- 数据生成 seed
- 文件 hash manifest

数据生成后冻结。后续所有 agent 都只读 task package。

人员分工

建议 4 到 5 人。每个人主要负责一组 arm 的 benchmark，并保证自己的 run root、日志和 summary 完整保存。

推荐分配

人员	负责 arm	主实验 run 数	稳定性/补跑	run root 示例
P0	总协调 + 数据冻结 + 合并统计	不固定	统一安排重跑	runs/P0_coordination_YYYYMMDD
P1	gpt_codex , gpt_cellcompass	约 200	GPT CellCompass 稳定性 20	runs/P1_gpt_YYYYMMDD

人员	负责 arm	主实验 run 数	稳定性/补跑	run root 示例
			datasets × 4 extra repeats	
P2	deepseek_cc , deepseek_cellcompass	约 200	DeepSeek 小子集 repeat	runs/P2_deepseek_YYYYMMDD
P3	mimo_cc , mimo_cellcompass	约 200	MiMo 小子集 repeat	runs/P3_mimo_YYYYMMDD
P4	biomni + 汇总分析	约 100	失败 case 复核和最终图表	runs/P4_biomni_analysis_YYYYMMDD

如果只有 4 人，P0 和 P4 可以合并：一个人负责协调、BioMNI、合并 summary 和最终分析。

每个人必须保存的内容

每个操作员负责自己的 run root，不要写入别人的目录。每个 run root 至少要有：

- RUN_METADATA.md
- batch_manifest.json
- batch_summary.jsonl
- batch_summary.csv
- batch_runner_logs/
- 每个 run 的 runner_stdout.log
- 每个 run 的 agent_log.txt
- 每个 run 的 eval_results.json
- 每个 run 的 .runtime/

每天需要同步的信息

每个人每天同步一行状态即可：

人员：
负责 arm：
run_root：
完成 / 计划：
失败数量：
需要重跑：
provider/API 问题：
备注：

P0/P4 每天把所有人的 batch_summary.csv 合并成：

- merged_summary.csv
- failed_runs.csv
- leaderboard_by_arm.csv
- runtime_by_provider.csv

运行目录规范

每个操作员只写入自己的 run root，避免互相覆盖。

推荐目录：

```
cytobridge_output/dynbench_week1/  
  manifest/  
  datasets/  
  runs/  
    P2_gpt_YYYYMMDD/  
    P3_deepseek_mimo_YYYYMMDD/  
    P4_biomni_YYYYMMDD/  
  merged/  
  reports/
```

每个 run root 内保留 runner 默认结构：

```
<run-root>/  
  batch_manifest.json  
  batch_summary.jsonl  
  batch_summary.csv  
  batch_runner_logs/  
  synthetic/<scenario>/agent_<agent>_skills-on_<run_label>/  
    runner_stdout.log  
    agent_log.txt  
    eval_results.json  
    velocity_field.csv  
    growth_rates.csv  
    holdout_prediction.csv  
    per_cell_fate.json  
    perturbation_results.json  
    driver_genes.json  
    .runtime/
```

不得手动删除：

- batch_runner_logs/*.log
- runner_stdout.log
- agent_log.txt
- .runtime/
- eval_results.json
- manifest 文件

这些文件是复现和失败分析的最小证据。

Run 命名规范

推荐 run label 格式：

`<arm_id>_r<repeat_index>`

例子：

`gpt_cellcompass_r01`

`mimo_cc_r01`

`deepseek_cellcompass_r03`

summary 表必须包含：

- scenario
- arm_id
- agent
- harness
- provider_used
- model_used
- thinking_used
- repeat
- seed
- commit
- run_root
- output_dir
- status
- total_score
- M1_velocity 到 M6_grn
- runtime_sec
- error

推荐配置文件

建议新增 `benchmark/dynbench/arms.yaml` , 集中定义所有 arm。

示例:

```
arms:
  - arm_id: gpt_codex
    agent: codex
    harness: codex
    provider: openai-codex
    model: gpt-5.5
    thinking: medium

  - arm_id: gpt_cellcompass
    agent: cytobridge
    harness: cellcompass
    provider: openai-codex
    model: gpt-5.5
    thinking: medium

  - arm_id: deepseek_cellcompass
    agent: cytobridge
    harness: cellcompass
    provider: deepseek
    model: deepseek-reasoner
    thinking: medium

  - arm_id: mimo_cellcompass
    agent: cytobridge
    harness: cellcompass
    provider: xiaomi
    model: mimo-v2.5-pro
    thinking: high

  - arm_id: biomni
    agent: biomni
    harness: biomni
    provider: biomni
    model: native
    thinking: null
```

如果 runner 暂时还不支持 `arms.yaml` , 短期可以用 shell 脚本分别跑不同 arm, 但每个 arm 的命令必须写入 `run_command.md` 。

命令模板

主实验模板：

```
python benchmark/dynbench/run_batch_agents.py \  
  --scenarios <comma_separated_100_scenarios> \  
  --agents <agent_or_arm_list> \  
  --repeats 1 \  
  --parallel 3 \  
  --device cuda \  
  --timeout 3600 \  
  --run-root cytobridge_output/dynbench_week1/runs/<operator>_<arm_group>_YYYYMMDD
```

--parallel 应按单张卡容量设置。默认建议 3，确认稳定后可以改为 4。

稳定性分析模板：

```
python benchmark/dynbench/run_batch_agents.py \  
  --scenarios <comma_separated_20_stability_scenarios> \  
  --agents cytobridge \  
  --repeats 5 \  
  --parallel 3 \  
  --device cuda \  
  --timeout 3600 \  
  --run-root cytobridge_output/dynbench_week1/runs/P2_gpt_cellcompass_stability_YYYYMMDD
```

单场景 debug 模板：

```
python benchmark/dynbench/run_dynbench.py \  
  --scenario <scenario_name> \  
  --agent-type cytobridge \  
  --mode skills-on \  
  --device cuda \  
  --seed 42 \  
  --run-root cytobridge_output/dynbench_week1/debug/<debug_id>
```

复现记录要求

每个 run root 必须额外保存：

RUN_METADATA.md

内容模板：

Run Metadata

- operator:
- date:
- machine:
- gpu:
- conda env:
- repo path:
- commit:
- branch:
- command:
- arms:
- scenarios:
- repeats:
- parallel:
- timeout:
- provider accounts:
- notes:

禁止在文档中写明文 API key。只记录 key alias，例如：

```
xiaomi_key_A  
deepseek_key_B  
gpt_pro_account_1
```

每日检查流程

每天固定两次检查：中午一次，晚上一次。

检查命令：

```

python - <<'PY'
import pandas as pd
from pathlib import Path

roots = list(Path("cytobridge_output/dynbench_week1/runs").glob("*"))
rows = []
for root in roots:
    csv_path = root / "batch_summary.csv"
    if csv_path.exists():
        df = pd.read_csv(csv_path)
        df["run_root"] = str(root)
        rows.append(df)
merged = pd.concat(rows, ignore_index=True) if rows else pd.DataFrame()
print("rows", len(merged))
if len(merged):
    print(merged.groupby(["provider_used", "status"]).size())
    print(merged.groupby("provider_used")["total_score"].mean(numeric_only=True))
    merged.to_csv("cytobridge_output/dynbench_week1/merged/merged_summary.csv", index=False)
PY

```

每日必须产出：

- merged_summary.csv
- failed_runs.csv
- leaderboard_by_arm.csv
- runtime_by_provider.csv
- daily_status_YYYYMMDD.md

失败重跑规则

失败分三类：

Provider failure

例子：

- quota
- rate limit
- 401
- 429
- reasoning_content
- connection error

处理：

- 可以换 key、base URL 或 fallback provider。
- 重跑不算 agent 算法失败。
- 必须保留原失败日志。

Harness/output failure

例子：

- 缺输出文件
- schema invalid
- time axis 错
- latent/gene label 错
- verifier failed

处理：

- 标记为 harness failure。
- 如果是明确 runner bug，修 runner 后重跑该 arm 的 affected scenarios。
- 不要手动修 agent 输出后直接计分。

Scientific failure

例子：

- 输出合法但 M1 到 M6 分数低。
- GRN 或 growth driver 预测差。
- holdout distribution 差。

处理：

- 不重跑，除非属于稳定性子集。
- 进入 failure taxonomy 和 case study。

一周时间表

Day 0: 冻结

- 确认 commit。
- 生成或确认 100 个数据集。
- 写 scenario manifest 和 hash。
- 写 arms 配置。

- 每个操作员完成一个 smoke run。

Day 1: Pilot

规模：

$6 \text{ datasets} \times 7 \text{ arms} \times 2 \text{ repeats} = 84 \text{ runs}$

目标：

- 检查 runner 是否能 resume。
- 检查 provider 成功率。
- 检查平均 runtime。
- 检查输出 schema。
- 估算真实成本。

Day 2 到 Day 4: 主实验

规模：

$100 \text{ datasets} \times 7 \text{ arms} \times 1 \text{ repeat} = 700 \text{ runs}$

要求：

- 每天两次合并 summary。
- 每天更新失败重跑列表。
- 不临时改数据集。
- 不临时改 evaluator，除非全体确认是 evaluator bug。

Day 5: 稳定性和补跑

规模：

$20 \text{ datasets} \times \text{GPT-5.5 CellCompass} \times 4 \text{ extra repeats}$
 $10 \text{ datasets} \times \text{top 3 arms} \times 2 \text{ extra repeats}$

目标：

- 估计 run-to-run 方差。
- 检查 top arms 排名是否稳定。
- 补 provider failure。

Day 6: 统计分析

输出：

- overall leaderboard
- per-metric leaderboard
- paired win rate
- bootstrap confidence interval
- stability std
- failure taxonomy
- runtime/cost table

Day 7: 报告

输出：

- benchmark report
- reproducibility appendix
- run manifest archive
- top failure case studies
- recommended next fixes

预算

固定预算：

$2 \times \text{GPT Pro} = 2 \times 200 \text{ USD} = 400 \text{ USD}$

API 和运行 buffer：

项	建议预算
DeepSeek API	200 到 500 USD
MiMo API	200 到 500 USD
fallback / rerun buffer	300 到 600 USD
机器和存储	100 到 300 USD

建议总预算：

最低可行：800 到 1000 USD
稳妥执行：1500 到 2000 USD
保守上限：2500 USD

Day 1 pilot 后必须用真实 token、失败率和 runtime 重新估算预算。

结果分析指标

主表：

- mean total score
- median total score
- per-metric mean
- win rate by paired dataset
- failure rate
- timeout rate
- runtime median
- cost per successful run

稳定性：

- per dataset std
- coefficient of variation
- rank stability
- best/worst repeat gap

case study：

- high score example
- low score example
- provider failure example
- schema failure example
- time-axis or gene-label failure example

最终交付物

必须交付：

```
cytobridge_output/dynbench_week1/  
  manifest/  
    dataset_manifest.json  
    scenario_hashes.json  
    arms.yaml  
    frozen_commit.txt  
  runs/  
    ...  
  merged/  
    merged_summary.csv  
    leaderboard_by_arm.csv  
    per_metric_by_arm.csv  
    failed_runs.csv  
    runtime_by_provider.csv  
    stability_summary.csv  
  reports/  
    daily_status_YYYYMMDD.md  
    final_benchmark_report.md  
    reproducibility_appendix.md
```

归档原则：

- 原始 per-run 目录不压缩前不要删除。
- 如果需要压缩，按 run root 压缩，并保留 batch_summary.csv 在外层。
- 所有失败 run 的日志必须保留。

决策点

一周内不要无限扩张实验矩阵。出现时间或预算压力时，按以下顺序削减：

1. 先减少 provider 小规模复核。
2. 再减少稳定性子集数量。
3. 再减少 hard/adversarial 数据集数量。
4. 最后才减少主实验 arm。

不能削减：

- 主实验每个 arm 至少一次。
- 日志保存。
- 数据和 commit manifest。
- public verifier 和 official eval。